



Programacion Orientada a Objetos **2**

Ing. Jose Rusca
Ing. Federico Stulich
Ing. Francisco De Grandis

Conceptos Fundamentales



1. Alcance de Variables
2. Funciones anónimas
3. Callbacks
4. Funciones flecha
5. Clausuras
6. Spread Operators
7. Manejo de arreglos

Variables - Alcance



Declaración de variables:

1. `var` -> Alcance a nivel de función
2. `let` -> Alcance a nivel de bloque
3. `const` -> Alcance a nivel de bloque

Variables - ejemplo var

```
function ejemploVar() {  
    var var1 = 10;  
    if(var1 === 10) {  
        var var2 = 40;  
  
        console.log("var1: " + var1 );  
        console.log("var2: " + var2 );  
    }  
  
    console.log("var1: " + var1 );  
    console.log("var2: " + var2 );  
}
```



Variables - ejemplo let

```
function ejemploLet() {  
    let let1 = 10;  
    if(let1 === 10) {  
        let let2 = 20;  
  
        console.log("let1: " + let1 );  
        console.log("let2: " + let2 );  
    }  
  
    console.log("let1: " + let1 );  
    console.log("let2: " + let2 );  
}
```

Va a producir un error en tiempo de ejecución.

Funciones anónimas



Son funciones sin nombre o identidad, que pueden ser utilizadas para ser pasadas como parámetros o en el caso que se tenga que retornar una función.

También conocidas como funciones lambda.

Funciones anónimas - con parámetro

```
var respuesta = function (nombre) {  
    console.log("Hola " + nombre + "! Soy una funcion  
anonima");  
}  
respuesta("Juan");
```



Funciones Flecha (Arrow Functions)



Una función flecha es una alternativa compacta a una función tradicional, pero con ciertas limitaciones.

Nos va a permitir escribir una función de forma más rápida y práctica pero debemos tener en cuenta los siguientes puntos.

Funciones Flecha (Arrow Functions)



Diferencias con las funciones clásicas:

1. La función flecha no define su propio contexto (this)
2. No puede ser utilizada como constructor (debido al punto 1)
3. Poseen un return implícito si no se define un bloque con llaves.

En la especificación del lenguaje vamos a encontrar más diferencias, pero las indicadas son las necesarias para el presente curso.

Funciones Flecha (Arrow Functions)

() => { ... instrucciones ... }

(a,b,c) = > {let res=a+b+c;
return res;}

(a,b,c) => a+b+c;

a => a + 10;



Funciones Flecha (Arrow Functions)

() => { ... instrucciones ... }

{
(a,b,c) => {let res=a+b+c;
return res;}

Con corchetes y valor de
retorno explícito.

(a,b,c) => a+b+c;

a => a + 10;

Funciones Flecha (Arrow Functions)

() => { ... instrucciones ... }

(a,b,c) = > {let res=a+b+c;
return res;}

{

(a,b,c) => a+b+c;

a => a + 10;

Sin corchetes y valor de
retorno implícito.

Funciones Flecha (Arrow Functions)

() => { ... instrucciones ... }

(a,b,c) = > {let res=a+b+c;
return res;}

(a,b,c) => a+b+c;

a => a + 10;

Con un parámetro
podemos omitir los
paréntesis.



Funciones anónimas - como parametro (callback)

```
setTimeout(function () {  
    console.log("Saludo en 1 segundo.");  
}, 1000);
```



Callbacks



Los Callbacks son funciones que se pasan como parámetros y van a ser utilizadas por la función.

Al igual que en el TimeOut, podemos definir un parámetro donde recibimos una funcion y la invocamos.

Callbacks

```
let presentacionFormal = (nombre, apellido)
```

```
=> "En este acto, presentamos a " + nombre + " " + apellido;
```



```
let presentacionInformal = (nombre, apellido)
```

```
=> "Bienvenido " + nombre + " " + apellido;
```

Callbacks

```
function presentarPersona(nombre, apellido, funcionPresentacion) {  
    console.log(funcionPresentacion(nombre, apellido));  
}  
  
presentarPersona("Juan", "Garcia", presentacionFormal);  
presentarPersona("Juan", "Garcia", presentacionInformal);
```



Clausuras (Closure)



Una clausura es una función que guarda referencias del estado adyacente.

Una clausura permite acceder al ámbito de una función exterior desde una función interior.

En **JavaScript**, las clausuras se crean cada vez que una función es creada.

Clausuras (Closure)



```
function creaFunc() {  
  
    var nombre = "Programacion Objetos 2";  
  
    function muestraNombre() {  
  
        console.log(nombre);  
  
    }  
  
    return muestraNombre;  
  
}
```

//llamamos a la funcion.

```
var miFunc = creaFunc();  
miFunc();
```

Clausuras (Closure) - Un ejemplo interesante



```
function creaSumador(x) {  
    return function(y) {  
        return x + y;  
    };  
}  
  
var suma5 = creaSumador(5);  
var suma10 = creaSumador(10);
```

//llamamos a la funcion.

```
console.log(suma5(2));  
// muestra 7  
  
console.log(suma10(2));  
// muestra 12
```

Sintaxis Spread



La sintaxis extendida permite a un elemento iterable tal como un arreglo o cadena ser expandido en lugares donde cero o más argumentos o elementos son esperados.

Puede ser utilizado en llamadas a función o arreglos literales

Sintaxis Spread - para pasar parametros

```
function sum(x, y, z) {  
    return x + y + z;  
}
```



```
const numbers = [1, 2, 3];  
console.log(sum(...numbers));
```

Sintaxis Spread - concatenando dos arreglos

```
var valores = [1,2,3];  
  
var numeros = [4,5,6];  
  
var totales = [...valores, ...numeros];  
  
console.log(valores);  
  
console.log(...valores, 4);  
  
console.log(totales);
```



Arreglos

Como crear arreglos, tres formas distintas:

```
let frutas = ["Manzana", "Banana"]
```

```
let frutas = new Array("Manzana", "Banana")
```

```
let frutas = new Array(10); //En este caso el  
arreglo no contiene items.
```

Arreglos



Vamos a ver varios métodos necesarios para el manejo de arreglos, muchos de ellos se utilizan pasando una función para el procesamiento de cada uno de los elementos del arreglo.

Arreglos



Métodos para el manejo de arreglos:

foreach

map

filter

includes

concat

find

findindex

indexOf

join

pop

push

reduce

shift

slice

splice

sort

Arreglos - foreach

El método **forEach()** ejecuta la función indicada una vez por cada elemento del array.

}

```
const array1 = ['a', 'b', 'c'];  
  
array1.forEach(  
    element => console.log(element)  
);
```

Arreglos - map

El método `map()` crea un nuevo array con los resultados de la llamada a la función indicada aplicados a cada uno de sus elementos.

}

```
var numeros = [1, 5, 10, 15];  
  
var dobles = numeros.map(function(x)  
{  
    return x * 2;  
});  
  
// dobles      [2, 10, 20, 30]  
  
// numeros     [1, 5, 10, 15]
```

Arreglos - filter

El método **filter()** crea un nuevo array con todos los elementos que cumplan la condición implementada por la función dada.



```
const words = ['spray', 'limit',  
'elite', 'exuberant', 'destruction',  
'present'];
```

```
const result = words.filter(word =>  
word.length > 6);
```

```
// Array ["exuberant",  
"destruction", "present"]
```

Arreglos - includes

El método **includes()** determina si una matriz incluye un determinado elemento, devuelve true o false según corresponda.

}

```
const pets = ['cat', 'dog', 'bat'];

console.log(pets.includes('cat'));
//true

console.log(pets.includes('at'));
//false
```

Arreglos - concat

El método **concat()** se usa para unir dos o más arrays. Este método no cambia los arrays existentes, sino que devuelve un nuevo array.

}

```
const array1 = ['a', 'b', 'c'];  
const array2 = ['d', 'e', 'f'];  
const array3 = array1.concat(array2);  
  
console.log(array3);  
  
//Array ["a", "b", "c", "d", "e", "f"]
```

Arreglos - find

El método **find()** devuelve el valor del primer elemento del array que cumple la función de prueba proporcionada.

}

```
const array1 = [5, 12, 8, 130, 44];

const found = array1.find(
  element => element > 10
);

console.log(found); //12
```

Arreglos - `findIndex`

El método `findIndex()` devuelve el índice del primer elemento de un array que cumpla con la función de prueba proporcionada. En caso contrario devuelve -1.

}

```
const array1 = [5, 12, 8, 130, 44];

const isLargeNumber =
  (element) => element > 13;

console.log(
  array1.findIndex(isLargeNumber)
);

// 3
```

Arreglos - indexOf

El método **indexOf()** retorna el primer índice en el que se puede encontrar un elemento dado en el array, ó retorna -1 si el elemento no esta presente.



```
var array = [2, 9, 9];  
  
array.indexOf(2);           // 0  
  
array.indexOf(7);           // -1  
  
array.indexOf(9, 2);        // 2  
  
array.indexOf(2, -1);        // -1  
  
array.indexOf(2, -3);        // 0
```

Arreglos - join

El método **join()** une todos los elementos de una matriz (o un objeto similar a una matriz) en una cadena y devuelve esta cadena.



```
const elements = ['Fire', 'Air',  
  'Water'];  
  
console.log(elements.join());  
  
// "Fire,Air,Water"  
  
console.log(elements.join(' '));  
  
// "FireAirWater"  
  
console.log(elements.join('-'));  
  
// "Fire-Air-Water"
```

Arreglos - pop

El método **pop()** elimina el último elemento de un array y lo devuelve. Este método cambia la longitud del array.

}

```
const plants = ['broccoli',  
'cauliflower', 'cabbage', 'kale',  
'tomato'];  
  
console.log(plants.pop());  
  
// "tomato"  
  
console.log(plants);  
  
// Array ["broccoli", "cauliflower",  
"cabbage", "kale"]
```

Arreglos - push

El método **push()** añade uno o más elementos al final de un array y devuelve la nueva longitud del array.



```
const animals = ['pigs', 'goats',  
  'sheep'];  
  
const count = animals.push('cows');  
  
console.log(count); // 4  
  
console.log(animals);  
  
// Array ["pigs", "goats", "sheep",  
  "cows"]
```

Arreglos - reduce

El método **reduce()** ejecuta una función reductora sobre cada elemento de un array, devolviendo como resultado un único valor.

}

```
const array1 = [1, 2, 3, 4];

const reducer =

    (accumulator, currentValue) =>
        accumulator + currentValue;

// 1 + 2 + 3 + 4

console.log(array1.reduce(reducer));

// 10
```

Arreglos - shift

El método **shift()** elimina el primer elemento del array y lo retorna. Este método modifica la longitud del array.



```
var peces = ['ángel', 'payaso',  
            'mandarín', 'cirujano'];  
  
var eliminado = peces.shift();  
  
console.log(peces);  
  
// payaso, mandarín, cirujano
```

Arreglos - slice

El método **slice()** devuelve una copia de una parte del array dentro de un nuevo array empezando por inicio hasta fin (fin no incluido). El array original no se modificará.



```
var nombres = ['Rita', 'Pedro',  
               'Miguel', 'Ana', 'Vanesa'];
```

```
var masculinos = nombres.slice(1, 3);  
  
// ['Pedro', 'Miguel']
```

Arreglos - splice

El método **splice()** cambia el contenido de un array eliminando elementos existentes y/o agregando nuevos elementos.



```
const months = ['Jan', 'March',  
  'April', 'June'];  
  
months.splice(1, 0, 'Feb');  
  
// Array ["Jan", "Feb", "March",  
  "April", "June"]  
  
months.splice(4, 1, 'May');  
  
// Array ["Jan", "Feb", "March",  
  "April", "May"]
```

Arreglos - sort

El método **sort()**
ordena los
elementos de un
arreglo (array)
localmente y
devuelve el arreglo
ordenado.



```
var valores =  
[1,3,45,1,23,25,1,8,25,1,9,10,25,23];  
  
valores.sort();  
  
//1,1,1,1,10,23,23,25,25,25,3,45,8,9  
  
valores.sort( (a,b)=> a - b );  
  
//1,1,1,1,3,8,9,10,23,23,25,25,25,45
```

Próxima Clase



Definición de Clases y Objetos en

JavaScript

